INFRANEX BT · OPERATOR HANDBOOK

# GPU Miner Setup Guide

From an empty dashboard to your first live Bittensor miner on subnet 64 (Chutes) with an RTX 4090 — every click, every command, every screenshot, in the exact order that keeps your costs at zero until the moment you choose to spend. Plus the no-GPU alternative: script mining on SN67 (Harnyx) from any CPU.

SN64 · CHUTES | RTX 4090 · 24 GB · $0.34/HR | CPU PATH · SN67 HARNYX | BINANCE FUNDING | REGISTER LAST | HOTKEY-ONLY SECURITY

Edition **v1.4** · September 14, 2026

Network **Bittensor Finney (mainnet)**

READ THIS FIRST

# The Golden Order

One principle runs the whole guide: **free things first, metered things last.** Your wallet costs nothing, your provider key costs nothing, the GPU meter starts the second a pod is created, and the registration burn starts your earning clock. Do them in this order and you never pay while you are still setting up.

LAPTOP · FREE

**Wallet**

coldkey + hotkey via btcli — keys never leave your machine

PLATFORM · FREE

**Provider key**

paste RunPod API key — connecting is free

WIZARD STEP 4 · COSTS

**Rent GPU**

pod created — $/hr billing starts now

WIZARD STEP 4

**Install**

automatic 10-step installer runs on the pod

WIZARD STEP 5 · COSTS

**Register**

burn fee → UID — earning clock + immunity start

ALWAYS ON

**Monitor**

daemon + alerts + P&L track everything

## Where the money moves

| Move | Amount | What it is |
|---|---|---|
| **Binance → coldkey** | burn fee + ~2 TAO | Plain deposit of funds. Nothing is burned; fully reversible by your own keys. |
| **coldkey → chain burn** | dynamic recycle fee | The actual burn — automatic inside the `btcli subnets register` command. SN64 burn-entry currently ≈ **0.0005 TAO**. |
| **card → RunPod** | $0.34/hr (4090) | GPU rent. Runs from pod creation until you **Terminate**. |
| **Coldkey mnemonic** | — | Never leaves your laptop. Never typed into any website, never stored by the platform. |

## Before you start — have these ready

- [ ] A laptop with **Python 3.9+** (for btcli — all wallet steps run here, not on the GPU)
- [ ] A **RunPod account** with billing set up, and its API key ready to paste
- [ ] **TAO on Binance** (or anywhere) — enough for the burn fee plus a ~1 TAO buffer
- [ ] **45–60 minutes** of uninterrupted time from the moment you click *Rent*
- [ ] Paper and a pen — the coldkey mnemonic is shown **once** and must be written down

PART A · ~5 MINUTES · $0

# Platform Prep — Log In & Connect RunPod

Two goals here: get into the platform, and hand it your RunPod API key so it can rent on your behalf later. Neither step costs anything — the key is only used when you actually create a pod.

### A1 Open the app and sign in

**Open** the preview link, then sign in with your operator ID and access code.

**Login screen** → enter **User ID** + **Access code** → **Sign in**

You land on the **Dashboard** — the chain badge (top right, green dot) should read *LIVE · FINNEY CHAIN* with a recent block number. That is your confirmation the platform is reading live chain data.

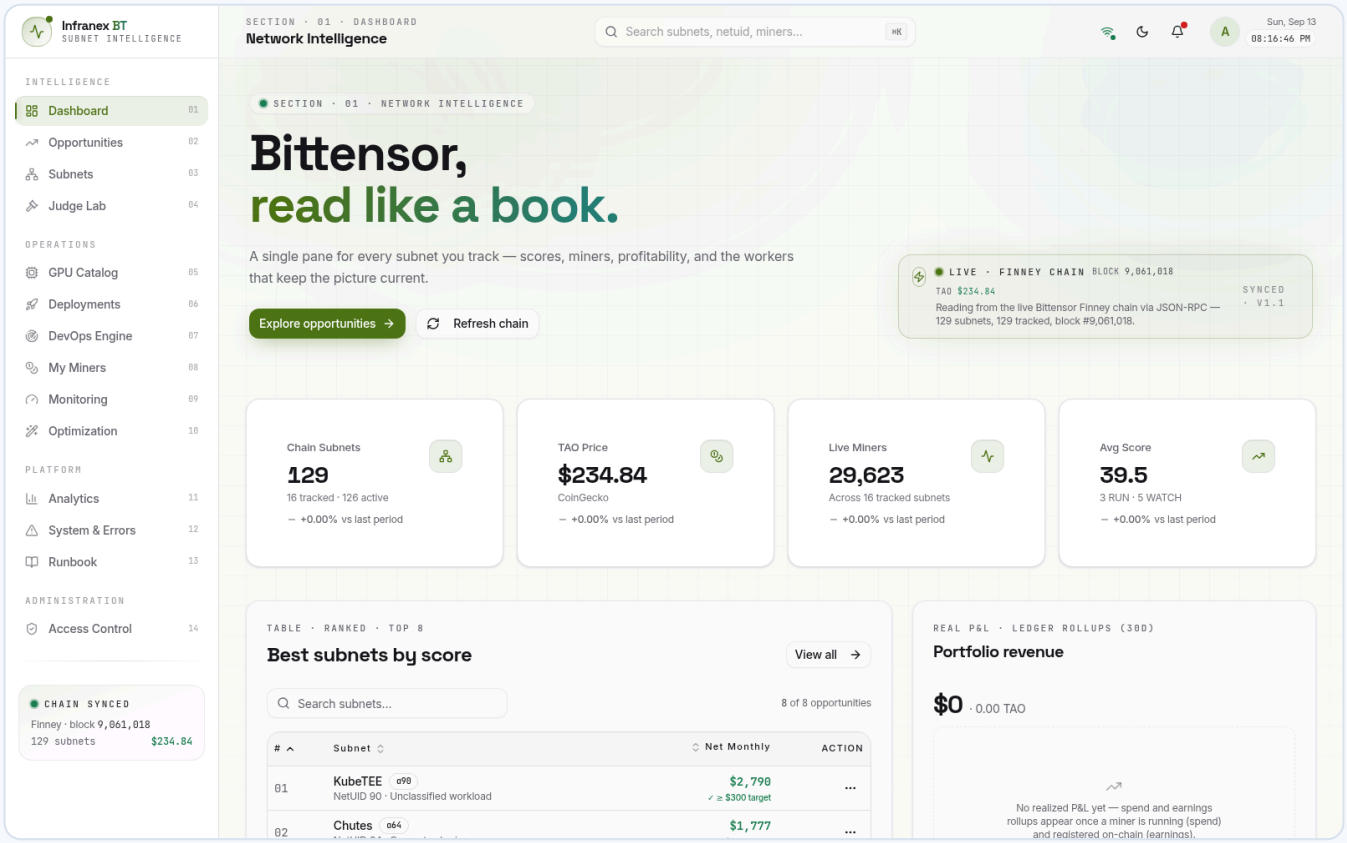



**Fig A-1.** The sign-in gate. Sessions are per-device; every sign-in is recorded in the audit trail (System & Errors → Audit trail).

## A2 Connect your RunPod API key

**GPU Catalog 05** → **Provider API keys** → paste key into the **RunPod** field → **Save & verify**

Get the key from `runpod.io → Settings → API Keys → Create API Key`, then paste it here. The platform validates the key on the spot and encrypts it at rest before storing anything. When validation passes you will see the green *saved* state on the RunPod row, and the GPU offer list (later steps) shows live prices.

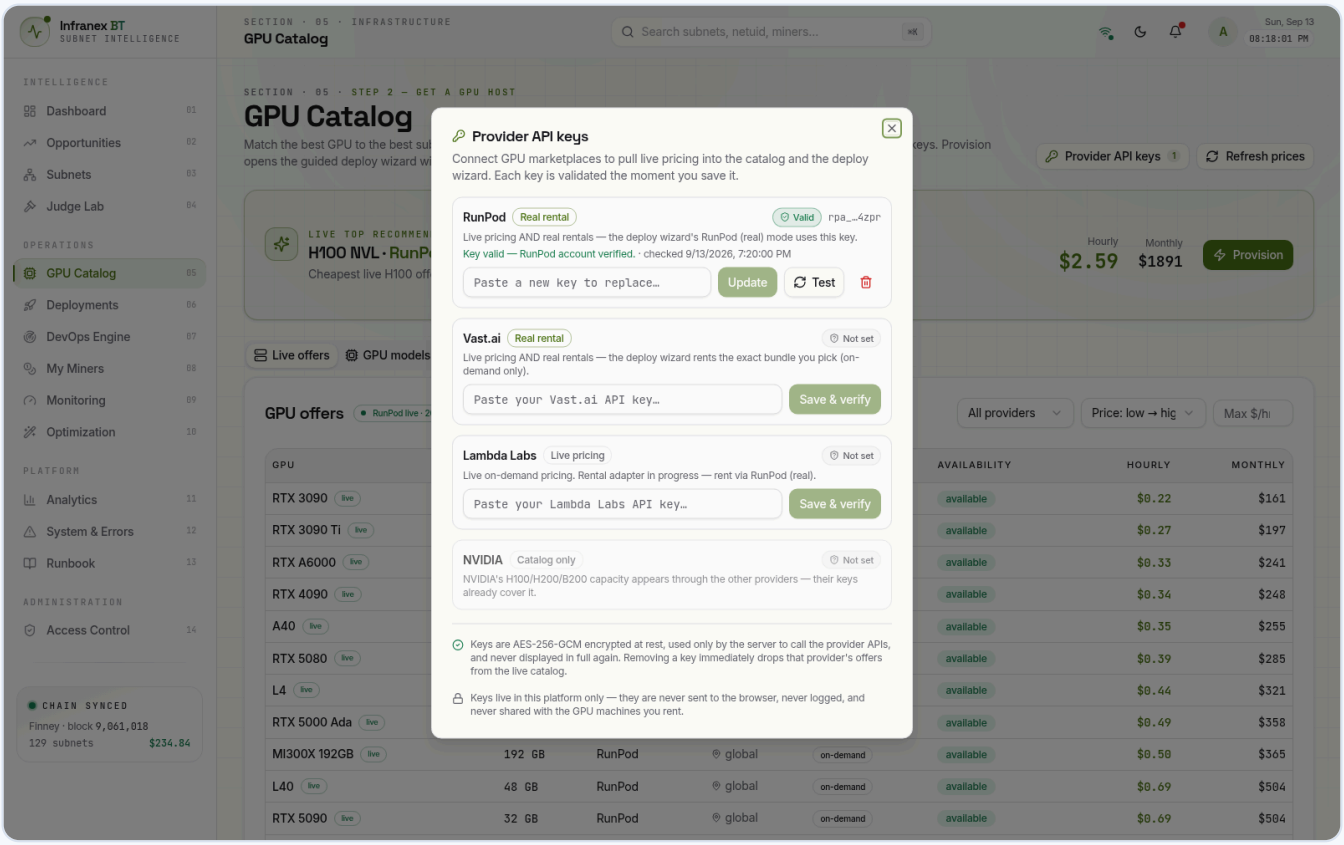



**Fig A-2.** The Provider API keys dialog — one encrypted slot each for RunPod, Vast.ai and Lambda Labs. Only the run-time platform can decrypt these; the UI never shows a saved key again.

**Why RunPod first?** The wizard's GPU step lists offers from every connected provider. With RunPod connected you will see the live RTX 4090 at ≈ **$0.34/hr** (prices move — check at rent time). Vast and Lambda can be added later; more providers just means more offers to choose from.

## A3 Optional now, recommended: save your wallet profile

**System & Errors 12** → scroll to **Wallet profiles** → **Add profile**

Save a **label** plus the btcli **wallet name** and **hotkey name** you plan to use (example used throughout this guide: wallet `infranex`, hotkey `miner`). This is public metadata only — it makes the deploy wizard prefill your wallet name instead of you typing it every time. The platform **cannot store mnemonics or private keys**; the API rejects any attempt.

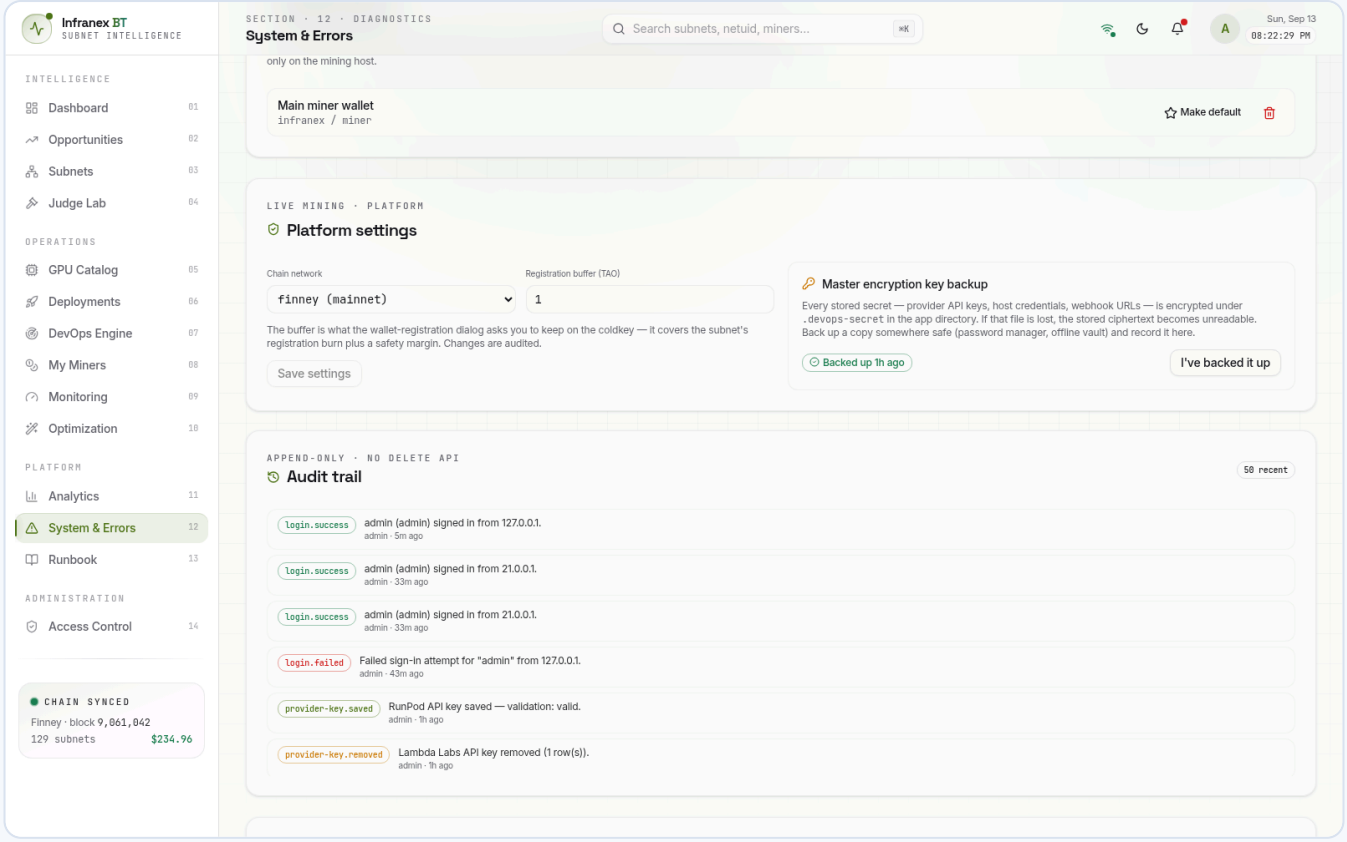



**Fig A-3.** System & Errors — bottom section. Wallet profiles (public metadata only), Platform settings (chain network + registration buffer), Master-key backup acknowledgment, and the append-only Audit trail.

PART B · ~15 MINUTES · $0 (ON YOUR LAPTOP)

# Create the Wallet & Fund It

The platform opens this same wizard for you later (Deployments → *Start wallet setup*) with every command generated and a **copy** button on each. You can follow either — the commands are identical. Do these four steps **before** renting the GPU, so the billing clock never runs while you set up keys.

### B1 Open the Wallet & registration wizard (optional shortcut)

**Deployments 06** → Mining Journey card → **Start wallet setup**

The wizard has 8 steps. Steps 1–4 happen on your laptop and cost nothing; steps 5–8 come back to the platform after the GPU is rented. Everything below walks the same eight steps with the exact commands.

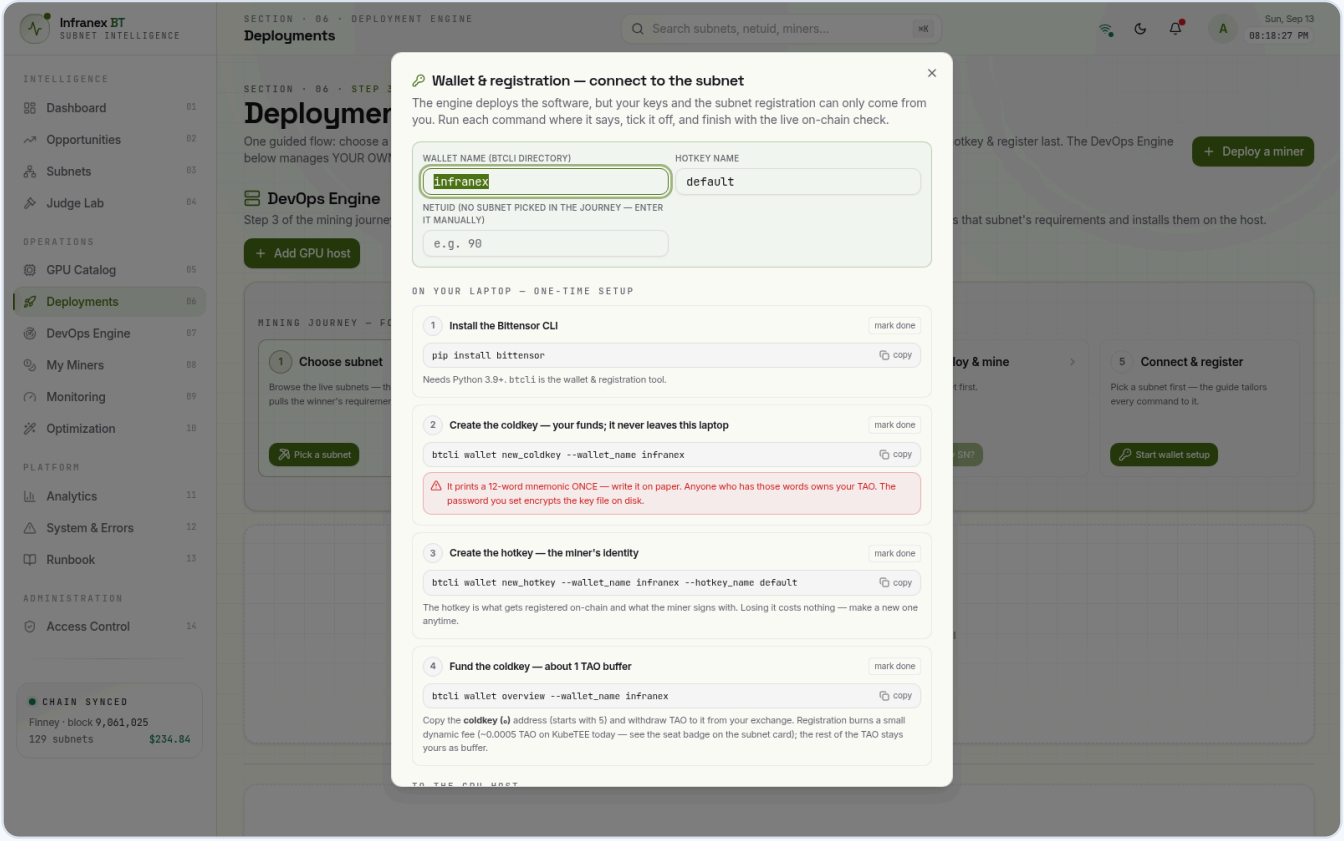



**Fig B-1.** The wizard's laptop section — wallet name, hotkey name, and steps 1–4 with copy-paste commands. The red warning is the one that protects your life savings: the mnemonic prints once.

## B2 Install the Bittensor CLI

On your laptop terminal (Python 3.9+ required):

YOUR LAPTOP · TERMINAL — STEP 1 OF 8

```
$ pip install bittensor
Successfully installed bittensor-9.x.x substrate-interface-1.x.x ...
$ btcli --version
btcli v9.x.x
```

### B3 Create the coldkey — your funds

YOUR LAPTOP · TERMINAL — STEP 2 OF 8

```
$ btcli wallet new_coldkey --wallet_name infranex
Wallet dir: /home/you/.bittensor/wallets/infranex
IMPORTANT: Store this 12-word mnemonic safely.
It is the ONLY way to recover your funds if you lose access:
1 word    2 word    3 word    4 word
5 word    6 word    7 word    8 word
9 word    10 word   11 word   12 word
Coldkey SS58 address: 5CA1abcdef...your-coldkey-address
```

**The 12 words are your money.** Write them on paper (never screenshot, never cloud notes, never paste anywhere — including into the platform). Whoever holds those words holds your TAO. The platform has no field for them by design.

### B4 Create the hotkey — the miner's identity

YOUR LAPTOP · TERMINAL — STEP 3 OF 8

```
$ btcli wallet new_hotkey --wallet_name infranex --hotkey_name miner
Hotkey SS58 address: 5Grwabcdef...your-hotkey-address
```

The hotkey is what gets registered on-chain and what the miner signs with. Losing it costs nothing — you can create a new one anytime; only the coldkey is precious.

### B5 Fund the coldkey from Binance

First, print your addresses:

YOUR LAPTOP · TERMINAL — STEP 4 OF 8

```
$ btcli wallet overview --wallet_name infranex
Wallet: infranex   ✓- coldkey: 5CA1abcdef...your-coldkey-address
                   ℏ- hotkey:  5Grwabcdef...your-hotkey-address
```

Then in Binance: **Spot → Withdraw → TAO**. Full click-by-click in **Chapter 7**. Send to the **coldkey** address (the ₵ one, starts with 5), amount = burn fee + ~2 TAO headroom. Skip ahead if you want the funding details now — the rest of this guide assumes TAO is on the coldkey.

PART C · ~20–30 MIN · COSTS START AT C4

# Pick the Subnet & Rent the GPU

The platform gives you two doors into the same wizard: the **Mining Journey** card (recommended — it tracks your progress through all five steps) or **Deploy a miner** (the raw wizard). This guide uses the Journey.

**What about the DevOps Engine strip and its "Add GPU host" button?** That is the third thing on the Deployments page, and it belongs to a *different* path: machines **you already own or rented yourself** (a home GPU box, or a pod created on the provider's own website). **Add GPU host** just asks for the machine's SSH address / user / password, the engine connects over SSH and runs the same 10-step install pipeline on it. **You do not need it for this guide.** When you rent through the wizard (C4 → C5), the platform registers the rented pod itself, connects with provider-issued credentials, and streams the install log inside wizard Step 4 — the pod shows up as a managed deployment without you ever clicking *Add GPU host*. Rule of thumb: **rented here → touch nothing in the DevOps strip; own the box → Add GPU host.**

**Reading the subnet verdicts — RUN / WATCH / AVOID.** Every subnet in **Opportunities** carries a verdict computed from the Miner's Ledger score: mid-pack monthly USD *after* GPU rent (weight 30%), seat safety — slot pressure, reward concentration, burn, immunity (20%), alpha economics — price trend, liquidity, slippage (20%), earning reality — rewarded ratio, newcomer ramp (15%) and fit & feasibility (15%). The bands: **RUN** = score ≥ 60 (healthy mid-pack economics — green), **WATCH** = 40–59 (viable *with skill or cheap entry* — amber), **AVOID** = below 40, or expected net profit under your $300/mo target (forced AVOID — red). **WATCH is advice, not a lock**: the wizard will happily deploy a WATCH subnet today. But mid-pack on those subnets hovers near breakeven once rent is paid, and the registration burn is the same on every subnet — so a "try WATCH now, move to RUN later" plan pays the burn twice. **First miner → RUN subnet** (that is why this guide mines Chutes SN64); consider WATCH only when you have a specific edge — a cheaper GPU, existing code for that subnet's niche, or you deliberately want a low-cost experiment.

## C1 Choose the subnet — Chutes (SN64)

**Deployments 06** → Mining Journey step 1 → **Pick a subnet** → search `chutes` → **SN64 Chutes · RTX 4090 24GB**

The picker already knows which GPU matches: it literally offers *SN64 Chutes — RTX 4090 24GB*. Confirm with **Mine this subnet**. The journey card now shows *SN64 · Chutes · needs ≥ 24 GB VRAM* on step 1.

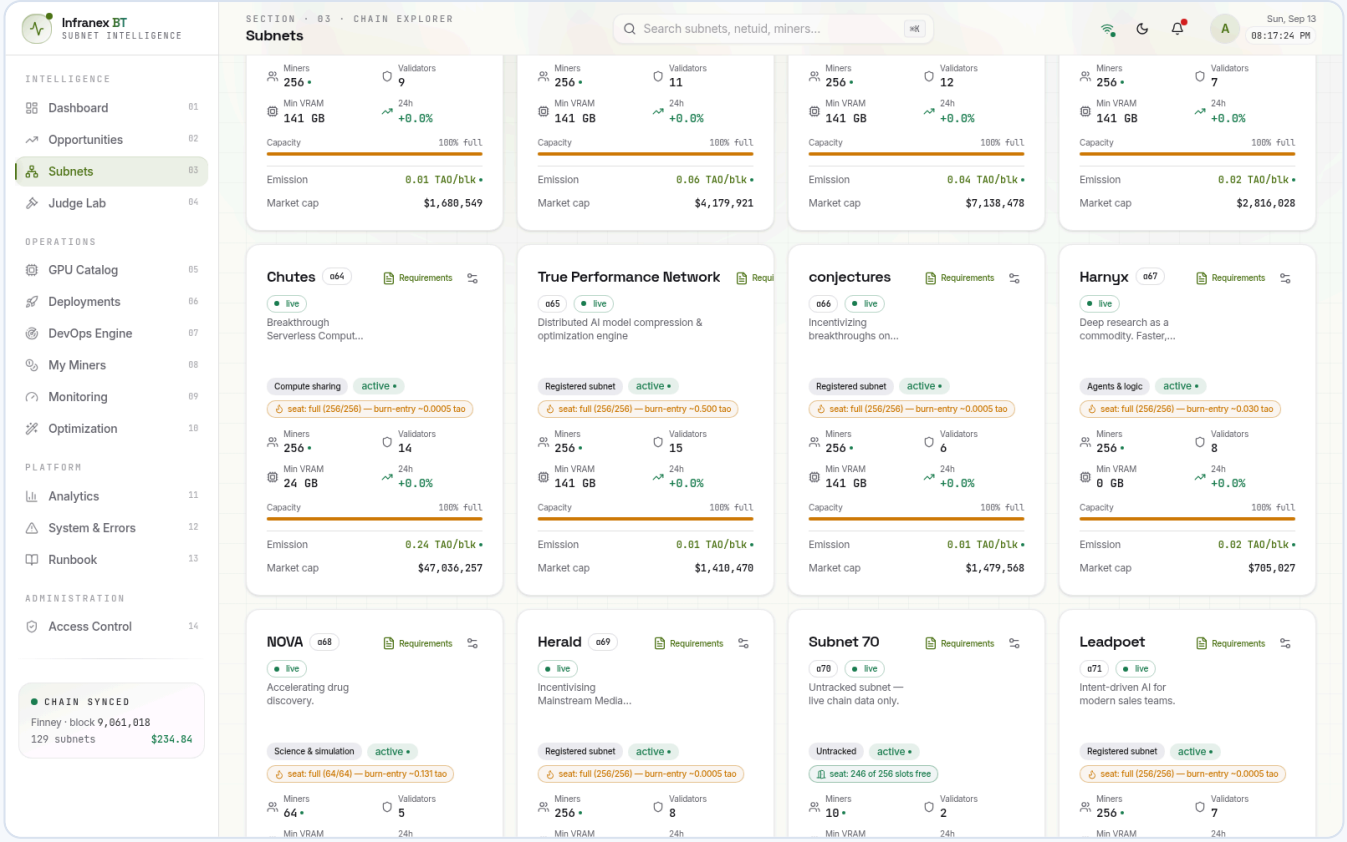



**Fig C-1.** Subnets view — the Chutes card carries the numbers that matter: **Min VRAM 24 GB** (a 4090 fits exactly), **burn-entry ≈ 0.0005 TAO** (your registration cost), 256 miners, emission 0.24 TAO/block. The Requirements button opens the full hardware profile.

## C2 Run the wizard — subnet → requirements → GPU

**Deployments 06** → **Deploy a miner** → Step 1 pick **SN64 Chutes** → Step 2 review requirements → **Find a GPU**

Step 2 shows the subnet's real requirements profile pulled live (what the miners on SN64 actually run). Step 3 lists **offers meeting the 24 GB requirement, cheapest first** — RTX 3090 at $0.22/hr, A6000 at $0.33/hr, **RTX 4090 24 GB at ≈ $0.34/hr**, and more.

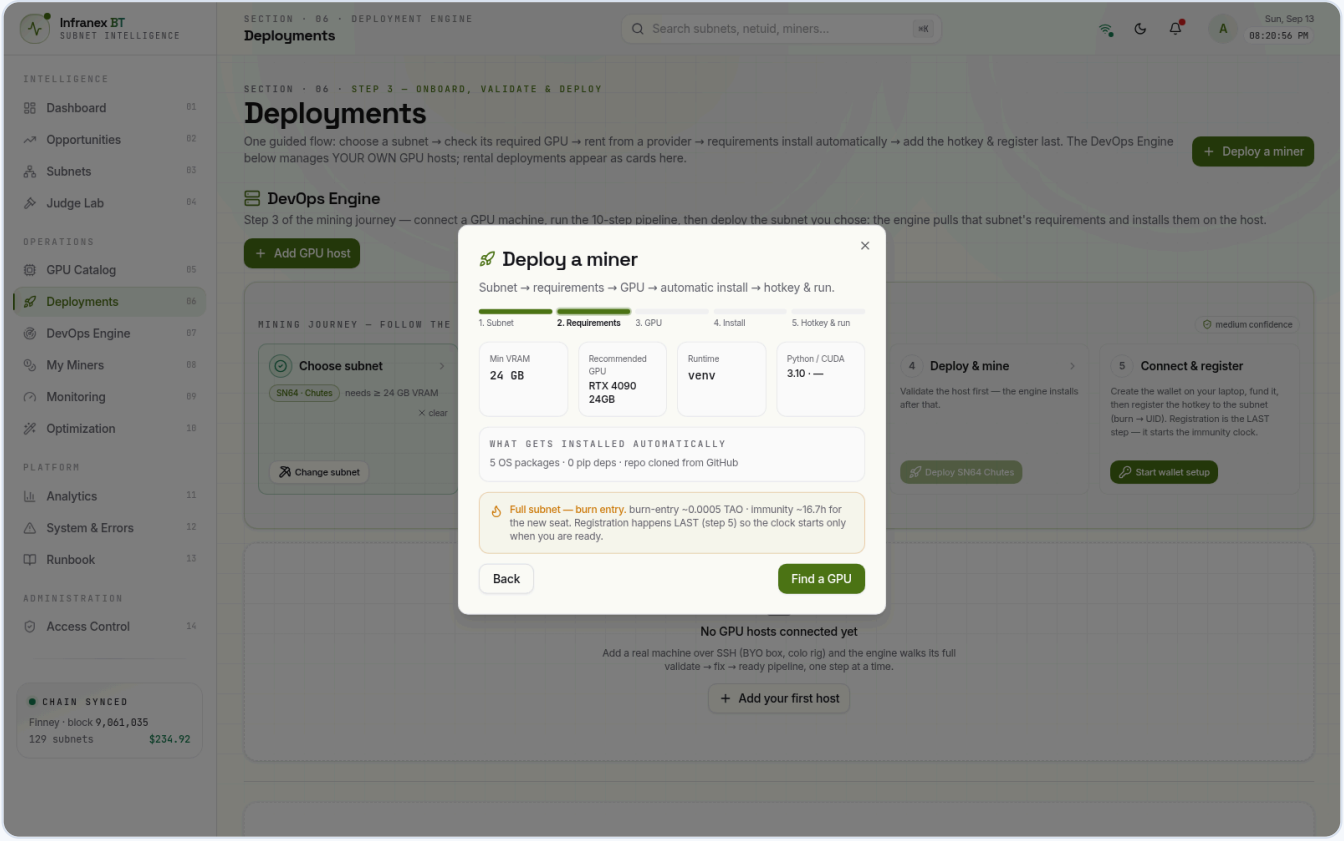



**Fig C-2.** Wizard step 2 — the requirements profile for SN64. With the journey subnet set, the wizard opens pre-selected here.

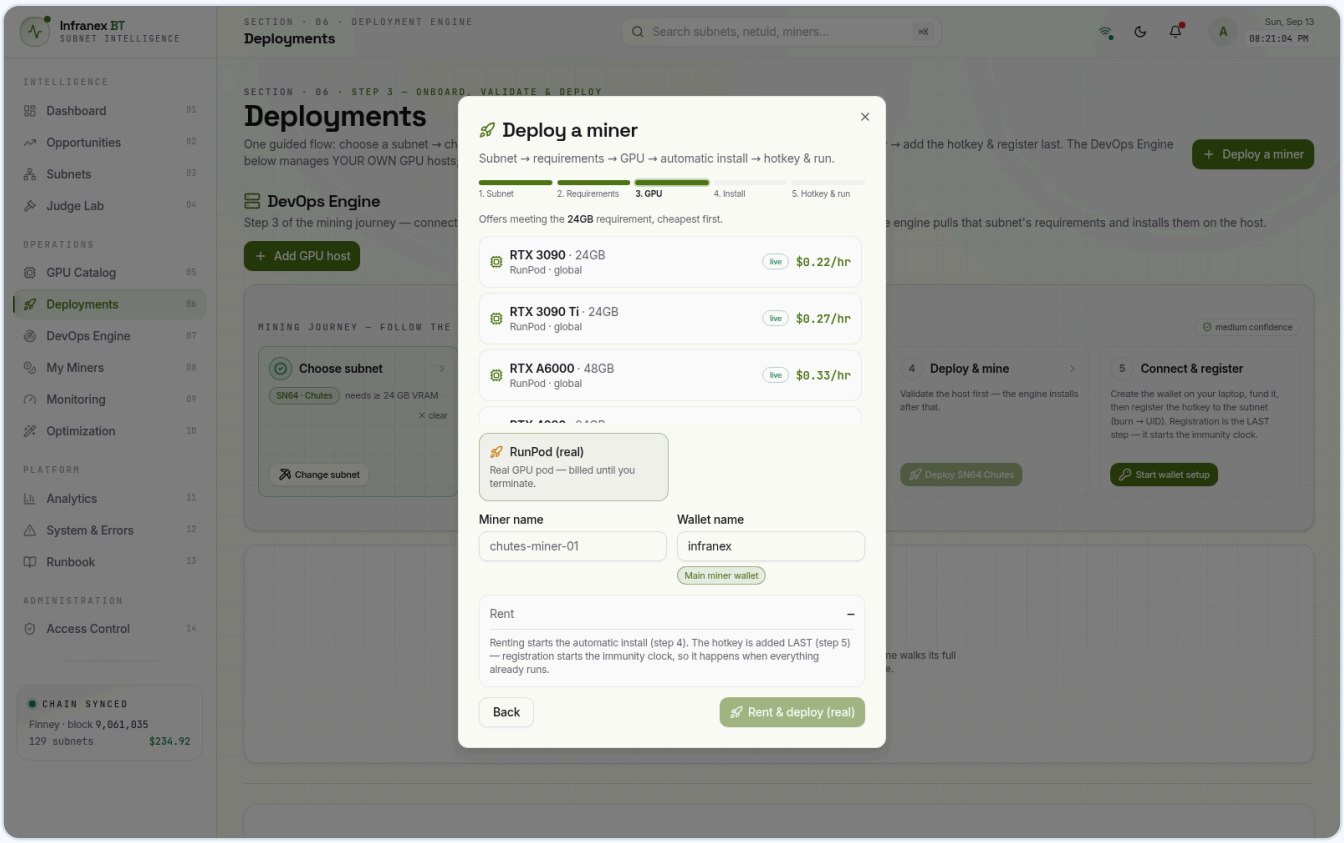



**Fig C-3.** Wizard step 3 — live offers filtered to ≥24 GB. Note the form below: miner name (auto: `chutes-miner-01`), wallet name prefilled from your saved profile (`infranex`), and the reminder that the hotkey is added LAST.

### C3 Confirm the GPU and the names

Select the **RTX 4090 · 24GB · RunPod** offer. Check the two name fields: **Miner name** is the label in your dashboard; **Wallet name** must match the btcli wallet you created in Part B (`infranex` in this guide — it prefills automatically from the wallet profile). Do not click the rent button until you are truly ready to start the meter.

### C4 Rent & deploy — the meter starts

Wizard step 3 → select **RunPod (real)** → **Rent & deploy (real)**

From this click: the platform creates the pod on RunPod (billing starts at the offer's $/hr), then runs the **automatic installer** — about ten steps shown live in the wizard's Install step: pull the image, write the subnet config, generate the pod's own SSH keypair, install the Node Daemon, start the miner process. You do nothing here except watch the log tick. When it finishes, the deployment card shows **started** and the wizard sits on step 5 waiting for your hotkey.

> **What "started" means now:** the miner process is running but serving *nothing yet* — it has no registered hotkey, so validators have nobody to score. This is by design: the expensive part (registration) waits until the cheap part (install) is proven working.

PART D · ~2 MINUTES · $0

# Put the Hotkey on the Pod

The miner on the pod needs the hotkey **keyfile** to sign messages. This file travels directly from your laptop to the pod over SSH — it never passes through the web app, and the coldkey file stays on your laptop.

### D1 Run the generated SCP command (wizard step 5)

Wizard step 5 → **"Copy the key files to the host"** → copy the `scp` command → run it on your laptop

The wizard generates the exact command with your pod's real IP, SSH port and path — it looks like:

YOUR LAPTOP · TERMINAL — STEP 5 OF 8

```
$ scp -P 12345 ~/.bittensor/wallets/infranex/hotkeys/miner
root@149.x.x.x:/root/.bittensor/wallets/infranex/hotkeys/
miner                              100%  496    12.3KB/s   00:00
```

Mark the step done (the *mark done* button) and move to step 6. Only the hotkey file crossed over; your coldkey and mnemonic are still exclusively on your laptop.

PART E · ~5 MINUTES · BURN HAPPENS HERE

# Register LAST — Burn, UID, Verify

Your miner is installed and healthy — **now** you pay the registration fee. The burn is not a transfer you make by hand: it happens automatically inside the register command, taken from your coldkey the moment you confirm. In exchange your hotkey gets a **UID** on SN64, and both the immunity clock and your earning window start.

**The gate before you burn:** the registration fee is the same on every subnet, so fail *cheap* first — run the free **Validator Lab** pre-flight (Part H, 10 minutes, $0) and confirm your spec scores **competitive or better** against SN64's judge. If it does, come straight back here and register. If it doesn't, Part H also shows you the highest-leverage fixes — much cheaper to apply them now than to discover a weak score after the burn.

## E1 Register the hotkey (wizard step 6, on your laptop)

Wizard step 6 → **"Register the hotkey"** → copy the command → run it on your laptop → confirm the fee

YOUR LAPTOP · TERMINAL — STEP 6 OF 8

```
$ btcli subnets register --netuid 64 --wallet_name infranex --hotkey_name
miner
Your balance:  2.0493   TAO
Recycle burn: 0.0005   TAO (subnet 64 entry cost)
Do you want to register? [y/n]: y
✓ Registered.
UID: 173   Netuid: 64   Block: 9,061,842
```

btcli always prints the exact current fee and asks for confirmation — nothing is spent until you type y. The fee is **per registration**: if the hotkey ever deregisters, re-registering costs the fee again. That is why the platform watches deregistration risk for you after this point.

## E2 Restart the miner so it serves its axon (wizard step 7 — one click)

Wizard step 7 → **Restart miner** button on the deployment card → the engine runs it over SSH for you

The miner announces its axon (IP:port) on-chain at startup — that announcement is how validators find and score you. The platform shows *restart pending* until this happens; one click on the card and the engine does the rest. Prefer the terminal? The wizard also prints the exact SSH command.

### E3 Verify on-chain (wizard step 8 — one paste, one click)

Wizard step 8 → paste your **hotkey SS58** (the ℏ address from `btcli wallet overview`) → **Verify**

The platform scans the live metagraph itself and reports:

```
PLATFORM · STEP 8 RESULT                                  CHAIN-VERIFIED

✓ REGISTERED — UID 173 on SN64 (Chutes)
active: yes      incentive: 0.0% (ramps up as validators score you)
emission: 0 rAO  block: #9,061,842
```

The only thing you ever paste into the platform is this **public** address — never a private key, never a mnemonic. Deployment status flips to **started · UID · registered**, the burn amount is recorded on the deployment, and the Mining Journey goes all-green.

PART F · FROM NOW ON · AUTOMATIC

## You Are Live — What Watches What

From the verified-UID moment, the platform's worker passes run every 90 seconds and you stop being a babysitter. Here is the map of what watches what, so you know where to look when something moves.

| Card / panel | What it tells you |
|---|---|
| **Dashboard — Portfolio revenue** | Real P&L: spend accrued every 90 s vs earnings rolled up daily from chain emissions, with a 14-day bar strip. Costs appear within minutes of the pod starting; earnings appear once validators start scoring your UID. |
| **Deployments — deployment card** | Status (started), UID, registration state, install log, the **Restart miner** and **Terminate** controls. |
| **My Miners — UID Defense** | Your UID's live chain vectors: incentive, emission, deregistration-risk sparkline. If your axon stops serving, this flags it with a runbook action before validators forget you. |
| **Monitoring / Runbook** | Alert rules (pod down, axon unreachable, registration lost, spend anomaly) — wire a Slack/Discord webhook in the channels dialog so they reach you. |
| **System & Errors — Audit trail** | Every credential change, approval, and sign-in, append-only. If something moved, it is written here. |

**Stopping vs terminating.** If you ever pause a miner, remember RunPod bills the pod's volume storage **even while stopped**. If you are done with a pod for good, use **Terminate** — otherwise an idle 100 GB volume quietly costs about $7/month forever.

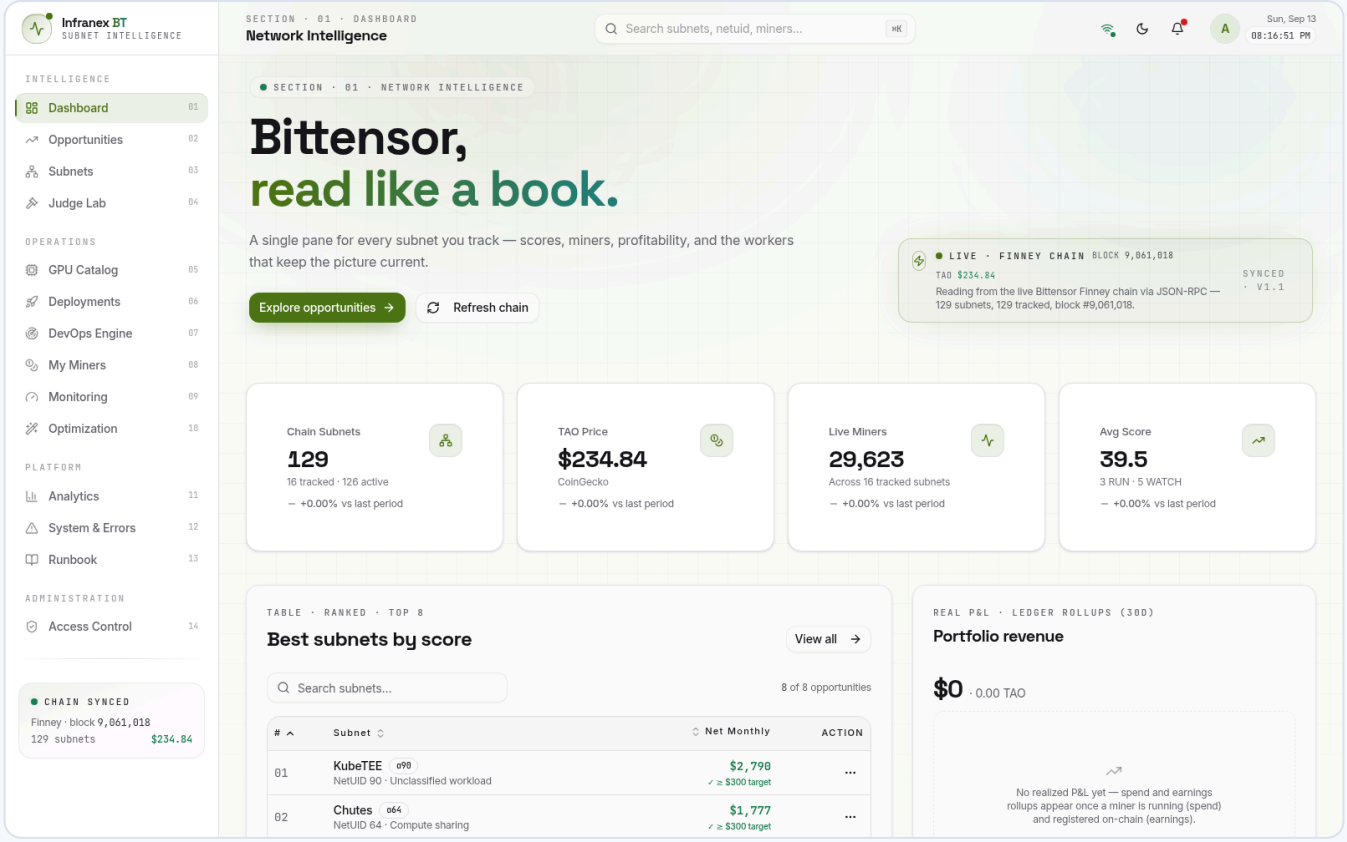



**Fig F-1.** The dashboard once you are live: chain badge, portfolio revenue (currently the honest empty state — it fills from the first accrual), and your tracked subnets with Chutes ranked 2nd by score.

PART G · YOUR JOB NOW · ~5 MIN/DAY

# Living as a Miner — The Operator Routine

Once deployed, the machine lifecycle runs on rails: **provisioning → installing (10 steps) → started → registered → earning**. "Started" alone earns nothing — only "registered" puts your hotkey on the metagraph so validators can score you. From that moment your job is not babysitting a terminal; it is reading five surfaces every day and acting on alerts. The platform watches every 90 seconds — **you decide**.

**Pick a posture first.** DevOps Engine 07 tags every miner with a strategy mindset, shown on the Strategy Board: **DEFEND**, **STEADY**, **EARN MORE**, or **OPTIMIZE**. For your **first 1–2 weeks run DEFEND** — your only goal is keeping the UID alive, because the burn is expensive to repeat. Once the UID survives a full week with nonzero incentive, move to STEADY, and only then consider EARN MORE (push throughput) or OPTIMIZE (cut $/hr for the same output).

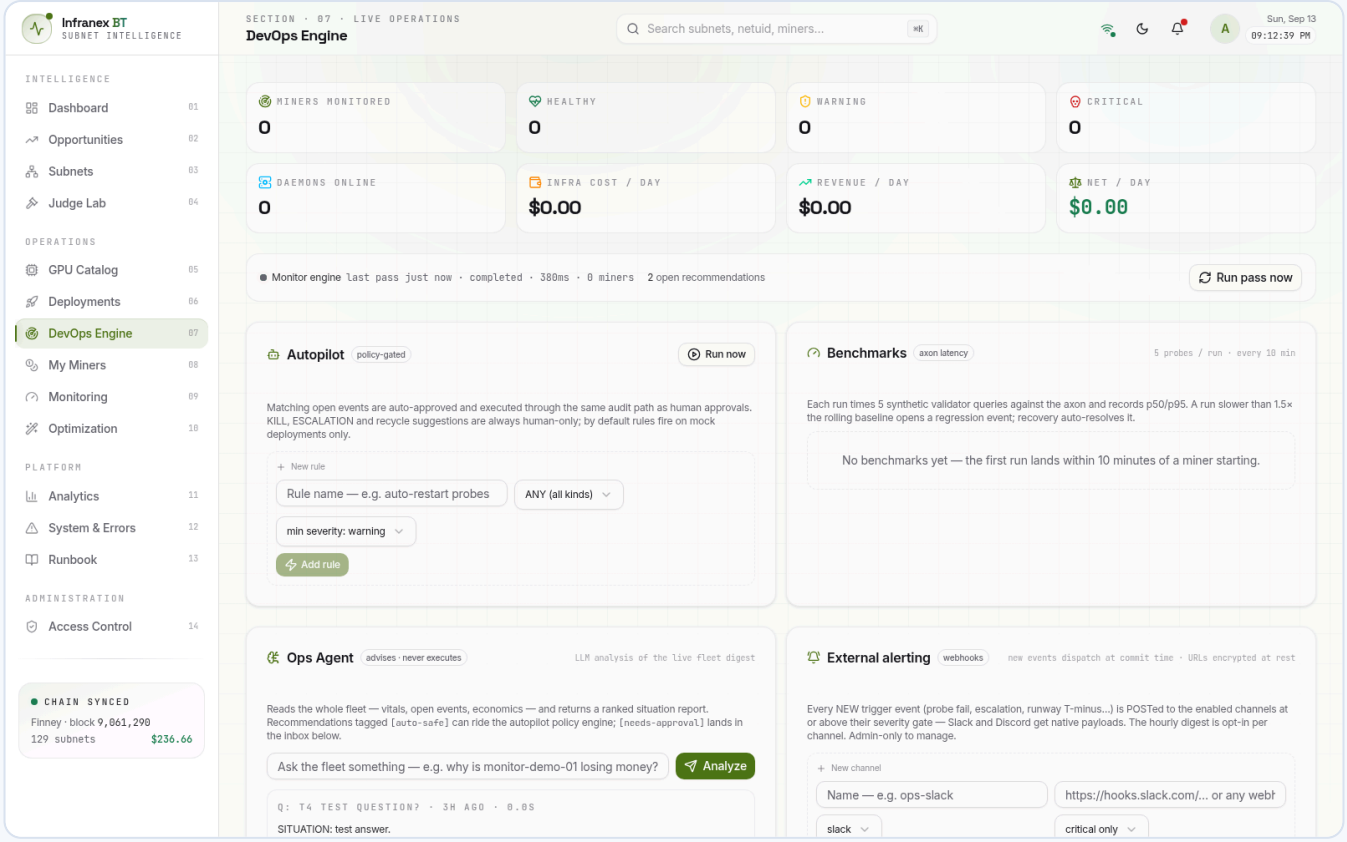



**Fig G-1.** DevOps Engine 07 — the automation deck. Counters for healthy/warning/critical miners, the 90-second monitor pass line, policy-gated Autopilot, axon latency benchmarks, the Ops Agent situation reports, and the External alerting panel where a real Slack/Discord webhook plugs in.

### G-1 Every day — read the deployment card

**Deployments 06** → your deployment card

The status line must read **started · UID n · registered** with all ten install steps ticked green. *started* without a UID means registration never landed — reopen the wallet wizard and run the register step again. A *stopped* card usually means the pod died or the RunPod balance hit zero — fund the account and press **Restart miner**; the engine does the SSH work in one click.

## G-2 Every day — check UID Defense is *healthy*

**My Miners 08** → UID Defense panel → risk level + sparkline

The engine scores your UID against the live metagraph on every pass and raises exactly six risk codes. Learn them once — they are the complete map of how a miner gets hurt:

| Risk code | Meaning → your action |
|---|---|
| **NOT_REGISTERED** *critical* | Hotkey missing from the metagraph → re-run registration. |
| **ZERO_INCOME** | Registered but earning 0 → normal in the first hours; if it persists past a day, restart and check the miner version. |
| **EVICTABLE** | Subnet at capacity and your immunity expired — you are a cut candidate → act fast: restart, update, improve; do not ignore this one. |
| **UNDERPERFORMING** | Scoring below your peers → update the miner revision; check subnet news. |
| **INCENTIVE_COLLAPSE** | Your incentive cratered → first check whether the whole subnet dipped (Subnets 03) or just you. |
| **PERSISTENT_DECLINE** | Days of falling incentive → consider a fresh deploy, a different GPU, or a different subnet. |

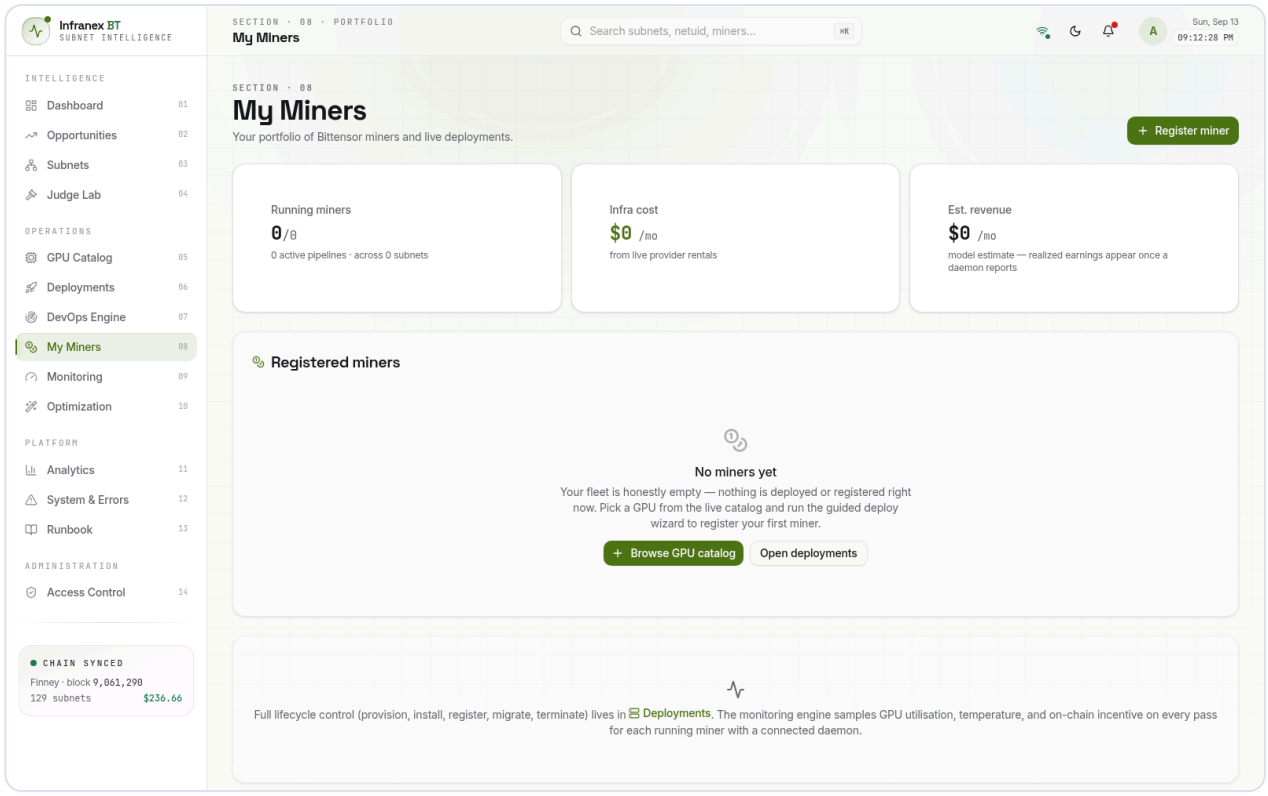



**Fig G-2.** My Miners 08 — the portfolio: running miners, monthly infra cost, estimated vs realized revenue, and the Registered miners list where UID Defense lives. Honest empty state here — yours fills the moment the first deployment registers.

## G-3 Every day — read the Monitoring rows

**Monitoring 09** → miner row → GPU util · temp · incentive

The engine samples each running miner every 90 seconds. **GPU utilisation** should spike regularly while the miner serves queries (60–95% bursts are normal) — a sustained flat 0% for hours means you are not being queried: check registration and version. **Temperature** is fine up to roughly 83–85 °C; sustained readings above that risk throttling — restart and re-check. **Incentive** (e.g. `I: 0.0042`) is your validator score: small daily growth is the goal. The four header cards give the money view: **Daily Emission** (at the live TAO/USD price), **Monthly Cost** (GPU rental), and **Net Profit/mo with ROI%** — for example `Net +$4.10/mo · ROI 61%`.

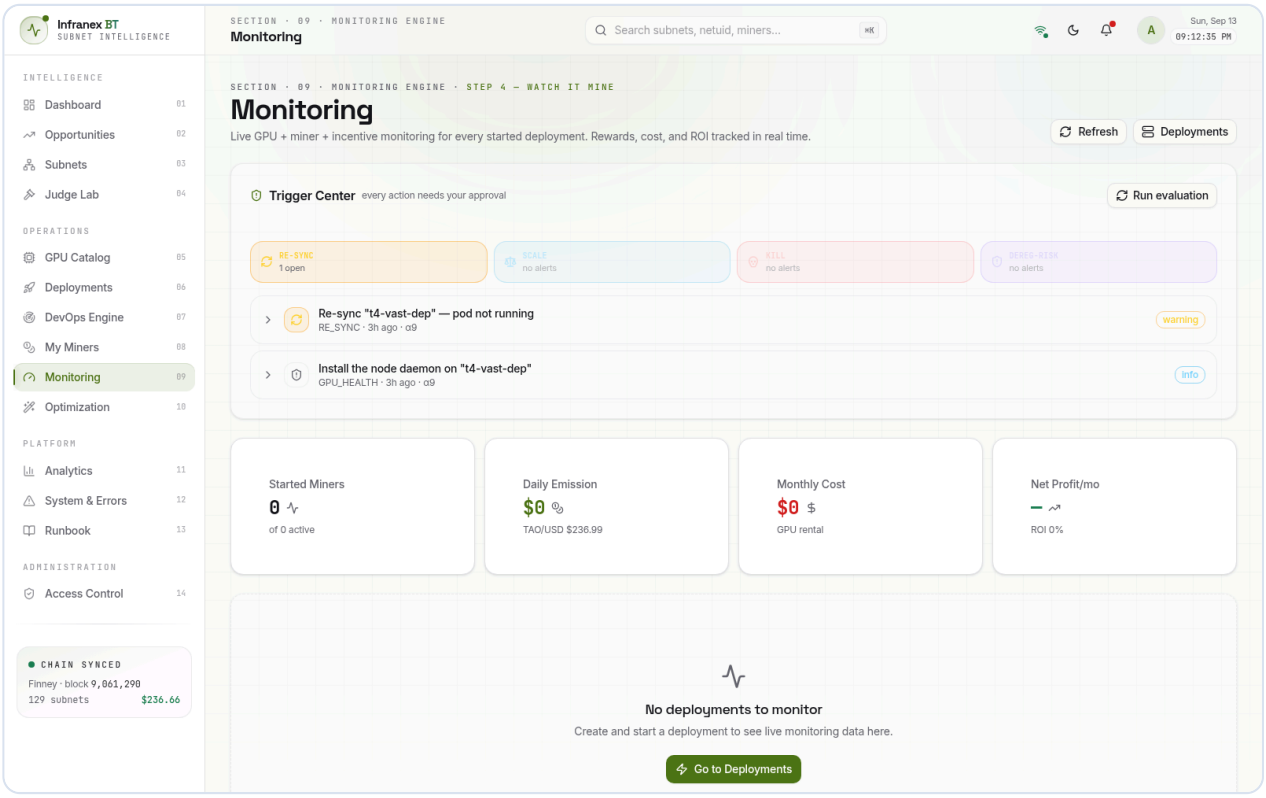



**Fig G-3.** Monitoring 09 — Trigger Center on top (every action needs your approval; here one RE-SYNC warning from a pod that is not running), then the money cards: Started Miners, Daily Emission at TAO/USD $236.99, Monthly Cost, Net Profit/mo with ROI.

## G-4 Every day — glance at the P&L card

**Dashboard 01** → Portfolio revenue card

Spend accrues within minutes of the pod starting; earnings roll up daily from chain emissions once validators score you. A healthy first week looks like: spend climbing steadily (about $8/day at $0.34/hr) while earnings start near zero and inflect upward as the UID earns trust. A card that only ever climbs on the cost side after a week of registration is your signal to revisit G-1 through G-3.

### G-5 Every day — keep 2–3 days of RunPod runway

An idle balance check takes ten seconds on the RunPod billing page. At $0.34/hr the burn is about $8.16/day, so keeping $17–25 of credit means never waking up to a stopped pod. If the balance reaches $0, RunPod stops the pod, the miner goes offline, and income stops — after topping up, one click on **Restart miner** brings it back. Put it on your calendar as a top-up reminder rather than trusting memory.

PART G.2 · WEEKLY & MONTHLY · ~15 MIN/WEEK

## The Deeper Cadence

| When | Where | What & why |
|---|---|---|
| **Weekly** | My Miners 08 | Compare incentive Mon→Mon instead of staring daily — weekly deltas are the real trend line. |
| **Weekly** | Subnets 03 → SN64 | Is the subnet itself healthy (emission, price, registration count)? A dying subnet drags every UID down — this is also where a better home would show up. |
| **Weekly** | Laptop terminal | Hotkey balance check with `btcli wallet overview` (example below). Move TAO hotkey→coldkey only once meaningful amounts accumulate — never rush this in week 1. |
| **Weekly** | RunPod console | Any *stopped* pods you are done with? **Terminate** them — an idle 100 GB volume costs ≈ $7/month forever. |
| **Weekly** | System & Errors 12 → Audit | Scan logins and credential events; anything you do not recognize = rotate the provider key now. |
| **Monthly** | GPU Catalog 05 | Re-check RTX 4090 pricing — your $0.34/hr moves, and a cheaper identical GPU elsewhere is a valid OPTIMIZE move (Vast offers appear alongside). |
| **Monthly** | Deployments 06 → revisions | Apply miner revision updates when flagged — Chutes refreshes its model list and stale miners lose incentive fast. |
| **Monthly** | Analytics 11 | Month-over-month earned vs spent vs ROI — the grow / hold / migrate decision, made on data. |

YOUR LAPTOP · TERMINAL — WEEKLY BALANCE CHECK

```
$ btcli wallet overview --wallet_name infranex
Coldkey     Hotkey   Network   Stake α     USD
infranex    miner    finney    0.0831      $19.44   ← growing weekly ✓
```

**What you never have to do manually:** spend accrual, GPU utilisation and temperature sampling, incentive sampling, UID Defense scoring, daily earnings rollups, and the append-only audit log all run on the platform's 90-second worker passes — and alerts dispatch to your webhook the moment a trigger fires. **The whole daily routine in one line:** deployment card green → UID Defense healthy → GPU% and incentive nonzero → P&L not bleeding → RunPod balance funded → done.

PART H · PRE-FLIGHT & ONGOING · $0 · ~10 MIN

# Validator Lab — Test Before You Burn

Every subnet has a **judge** — the validator that scores miners and decides your incentive. Judges are not alike: one pays for raw speed, another for output quality, another for the cheapest price. Validator Lab (nav **04**) reads the subnet's *actual validator code*, extracts what it rewards and how brutally it pays, then **simulates your miner spec against it** — all before you spend a rupee. It is the only place in this guide where failing is free.

### H-1 Mine the judge profile — SN64

**Validator Lab 04** → pick **α64 · Chutes** → **Re-mine profile**

The lab resolves the subnet's repo from on-chain identity, scans the validator source for scoring evidence (*24 evidence hits* for Chutes), and classifies the judge into an archetype. Chutes comes back as **Market Clearing** at **66% profile confidence** — the lab tells you how much code evidence it found, so you know how much to trust the model.

### H-2 Read the profile — what the judge pays for, and how brutally

Two things matter. **Composite weights** — for SN64: Price Competitiveness 58%, Throughput 17%, Response Speed 12%, Uptime 7%, Quality 5%, Resource Fit 2%. Translation: on Chutes, a cheap, fast miner beats a brilliant-but-pricey one. **Cohort brutality** — *Brutal 62/100*: 256 miners registered, only 17 earning (6.6% earn ratio), and the top 10% of earners take 30% of the incentive. This is the same "reward concentration" risk the Opportunities verdict prices in — here you see it raw.

### H-3 Set your real spec and run the simulation

In the **Mock-validator simulator**, dial the sliders to what your 4090 will actually run: **2.00s** response latency, **99.50%** uptime, **80%** quality, **25 units/s** throughput, **$0.50**/1M tokens. Then **Run simulation**. The scoring curves are deterministic — same spec, same result — so you can iterate freely.

## H-4 Read the verdict and the levers

The example run scores **62.1** — verdict **competitive**, estimated cohort percentile **~p82**, **1.36×** the cohort median. Below the numbers: a per-dimension breakdown (speed 0.98, uptime 0.90, quality 0.77 — price only 0.50, the weak axis on a Market Clearing judge) and the **highest-leverage fixes** ranked by composite points gained: *+14.5 cut asking price*, *+3.6 increase throughput*, *+0.7 resource fit*. Fix what is cheap to fix, re-run, and watch the composite move.

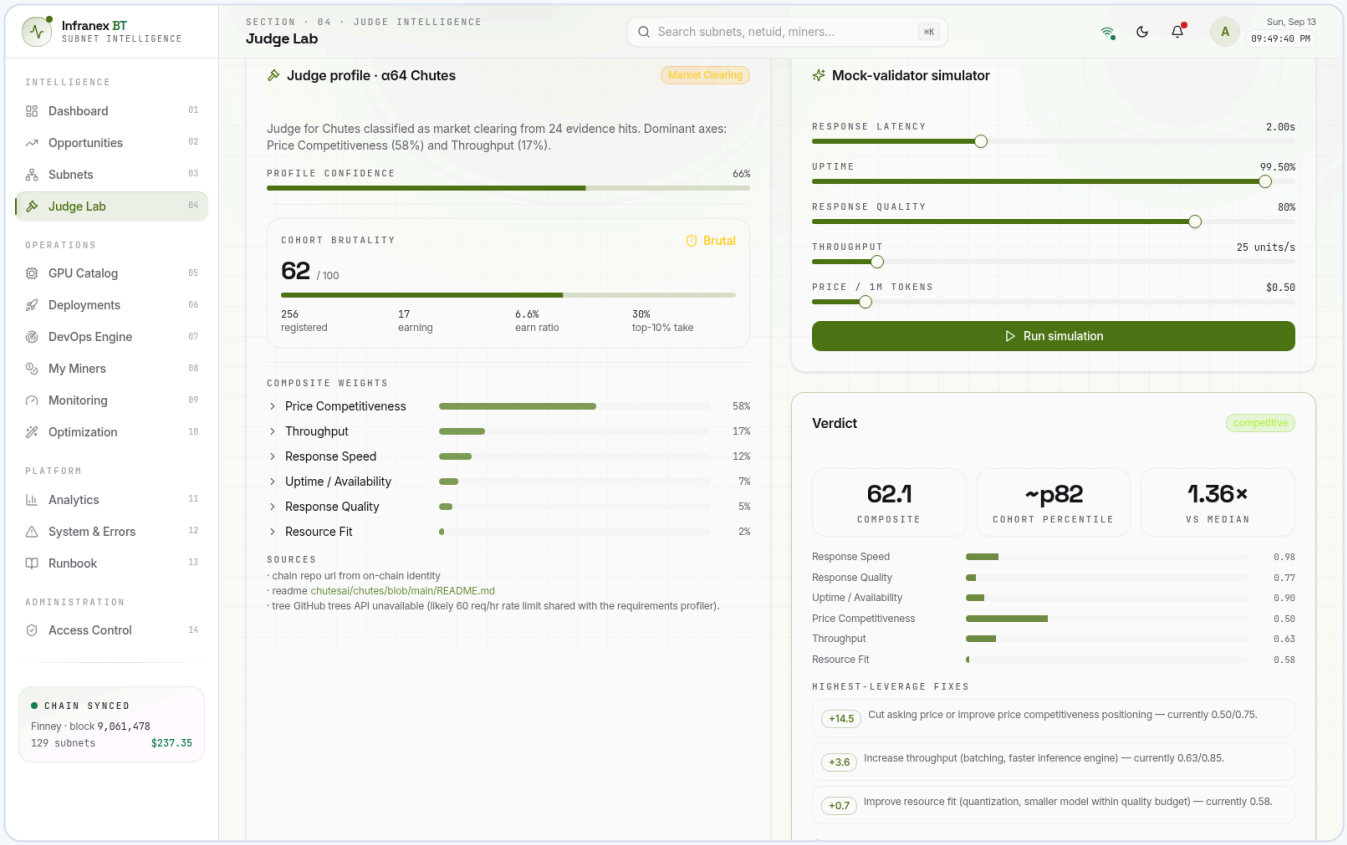



**Fig H-1.** Validator Lab on SN64 — left: the mined judge profile (Market Clearing, 66% confidence, brutality 62/100, composite weights with Price at 58%); right: the mock-validator simulator at the 4090 default spec and its verdict — composite 62.1, *competitive*, ~p82 percentile, 1.36× vs median — with the ranked highest-leverage fixes.

| Verdict | Composite | What it means for your burn |
|---|---|---|
| **strong** | ≥ 75 | Green light — top of the cohort; register with confidence. |
| **competitive** | 60–74 | **The pass bar** — viable mid-pack seat, above-median earnings. Register. |
| **marginal** | 45–59 | Apply the listed fixes first — burning now locks in a weak seat. |
| **weak** | < 45 | Do not burn on this subnet with this spec — re-spec or change subnet. |

**The pre-burn rule:** register only when the simulation says **competitive (≥ 60) *and* above 1.00× vs median** — that is the cheapest insurance this guide can buy, because the alternative way to learn your score costs the burn plus weeks of rent. **Honest caveat** (the lab prints it too): this is a deterministic model of the judge built from its code — not the validator itself. Confidence % tells you how strong the evidence was; calibrate with small stakes. **When to re-run:** before your first registration (the gate note in Part E), after any spec/config change, whenever Monitoring shows incentive drifting down, and as part of the Part G.2 weekly cadence — the judge's weights shift when the subnet updates its validator.

PART I · THE SECOND PATH · $0 HARDWARE · ONE SMALL BURN

# The CPU Path — Harnyx SN67, Script Mining

Everything up to here assumed a GPU. SN67 (Harnyx) is the second way in: **script mining**. Your miner is a Python agent script submitted to the subnet — validators execute it inside their own sandboxes and score the answers. There is no pod to rent, no CUDA to configure, no 24/7 box to babysit; the machine that matters is theirs, and a CPU is enough to build and submit. The app now ships a dedicated **CPU Guide** (nav **05**) that walks this path with live chain numbers and a per-user progress checklist — this part maps 1:1 to it. Entry is the cheapest seen on Finney: the registration burn floats around **0.05–0.08 τ (≈ $12–18)** against SN64's ~1 τ.

### I-1 Read the economics — why SN67 is the low-risk entry

**CPU Guide 05** → live strip: **SN67 economics**

The strip is the Opportunities scanner's own data, re-rendered as six live numbers. What they said at press time: **256/256 seats full** (burn is the entry path — the situation Part E describes), **217 miners earned** last epoch — an **~85% earn ratio**, the friendliest cohort in this guide by far (SN64 Chutes: 6.6%) — **~9.8 τ/day** flowing to miners, alpha at **0.0195 τ**, and a **top-10% incentive share of ~24%**: a wide spread where mid-pack miners keep their seats instead of feeding a whale. The base-case net from the scoring engine: **~$100/mo** (bear −$37, bull +$179). Re-read the strip weekly — if the alpha price bleeds or the spread tightens, the deal changed.

## I-2 The nine phases — the golden order, CPU edition

The CPU Guide view tracks your progress phase by phase (marks persist per signed-in user). The order is the same discipline as always — **register LAST** — but phases 0–5 cost nothing and need no hardware beyond the machine you already own:

P0 · FREE

### Size the deal

Opportunities filter "CPU-only" → SN67 scored 65.5, net ~$100/mo base case.

P1 · FREE

### Wallets

Coldkey + hotkey on your box; record the hotkey address in the Wallets view.

P2 · FREE

### Get the code

Clone harnyx/harnyx; install the harnyx-miner-sdk in a small venv.

P3 · FREE

### Study the champion

Sync the judge profile (netuid 67) in Validator Lab; probe the top miner via MCP.

P4 · FREE

### Build the agent

Targets: <30 s, ≤$0.50/1M, ≥25 tps, quality ~80% with citations.

P5 · FREE

### Local eval

Spar with Validator Lab — re-simulate after every design change.

P6 · COSTS

### Register

The ONE burn (~$12–18) — only on a green gate. btcli subnet register.

P7 · FREE

### Submit script

Harnyx CLI's job — the app deliberately has no upload path.

P8 · FREE

### Monitor & earn

Weekly strip check; MCP + Harnyx dashboard for scores; taostats for payouts.

### I-3 The gate — Part H's rule, SN67's numbers

Sync the profile (Validator Lab 04 →netuid **67**→**Re-mine profile**): SN67's judge is **Market Clearing** with **Price 56%** + Quality 19% composite weight, a **~30 s** response deadline, a **~$0.50/1M** price reference and **~25 tps** throughput reference. The Part H verdict table applies verbatim. The pre-flight that proved the rule on this very subnet: a slow-but-deep agent (240 s, $0.60/1M, 72% quality) scored **41.9 — weak**; a tuned fast+cheap agent (25 s, $0.45/1M, 80%, 25 tps) scored **62.1 — competitive ≈ p90, 1.87× the median**. Same spec, same judge model, two different decisions: don't burn, and clear to burn.

### I-4 The commands — four steps, the burn in the middle

YOUR LAPTOP · TERMINAL — CPU PATH · STEPS 1–4

```
$ git clone https://github.com/harnyx/harnyx.git && cd harnyx
$ python -m venv .venv && source .venv/bin/activate
(python 3.10+ is plenty — the SDK wraps the network protocol)
$ pip install -e harnyx-miner-sdk
build your agent inside the SDK harness · verify in Validator Lab (I-3)
$ btcli subnet register --netuid 67 --wallet.name harnyx --wallet.hotkey
sn67miner
burns ~0.05–0.08 τ — run ONLY when the gate says competitive or better
```

After registering, submit the agent through the repo's Harnyx miner tooling (the SDK's submission flow is the source of truth — it signs with your keys and hands the script to the validators). Your hotkey then earns on SN67 every epoch it clears the judge.

### I-5 Honest boundaries — what the app does NOT do on this path

The platform's execution layer stays GPU-first by design, and on this path that is a feature, not a gap. Three things the app deliberately does not do: **no CPU pod provisioning** — the Deploy Wizard rents GPU pods only (CUDA is hardcoded end-to-end), and script mining needs no rented box; **no script submission** — the app has no upload path and doesn't need one, that's the Harnyx CLI's job (I-4); **no earnings ledger for self-managed miners** — My Miners tracks deployments the app provisions itself, so a CLI-run miner is invisible there. The ground truth for this miner lives in three places: the app's strip for economics, Harnyx's own dashboard (dashboard.harnyx.ai/benchmark) + MCP for your agent's scores, and taostats for the coldkey's actual payouts.

**The one-line picture:** the app decides *where* to mine and *whether to burn*; your CPU builds the agent; the Harnyx platform runs it; the app keeps watching whether the deal is still good.

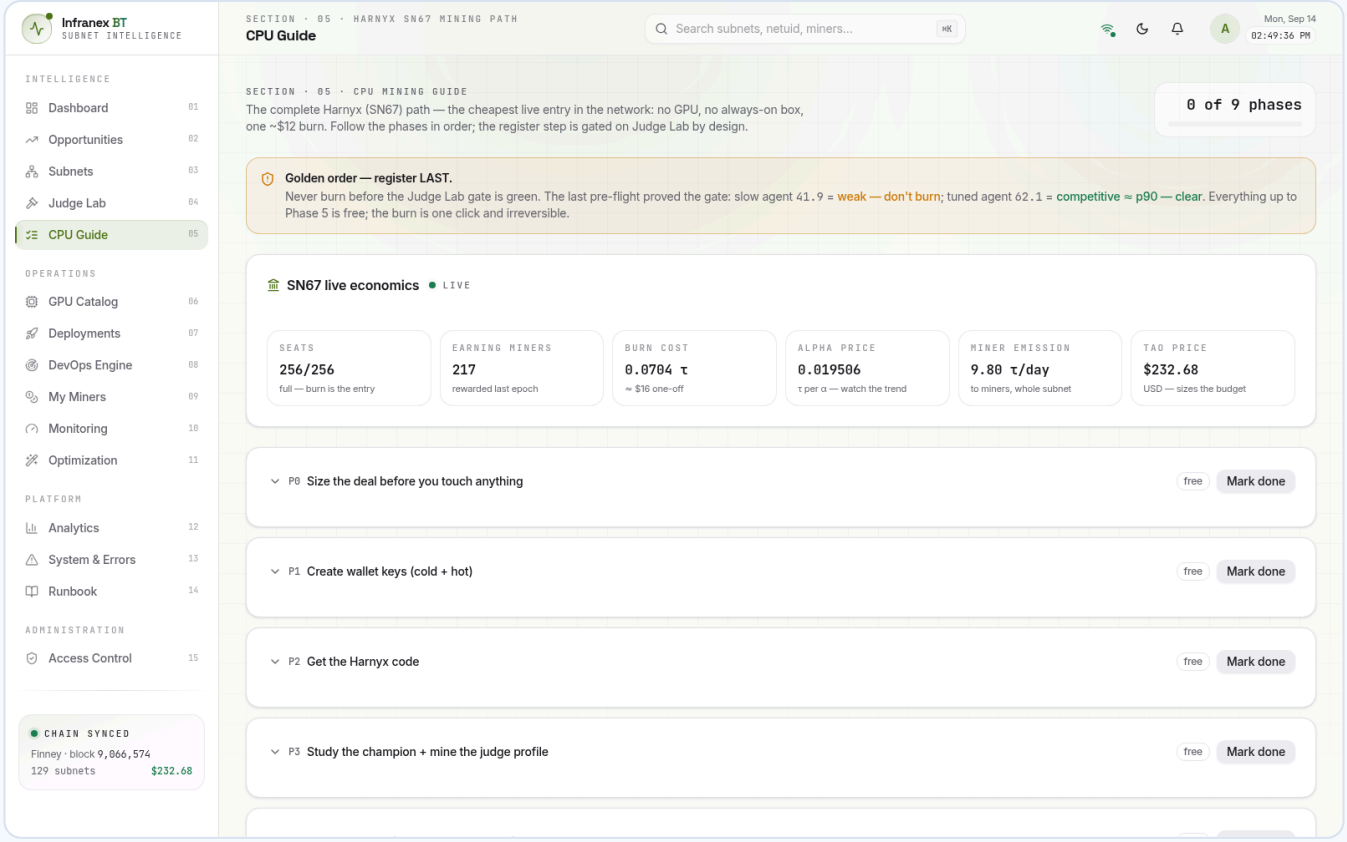



**Fig I-1.** The CPU Guide (nav 05) — top: the live SN67 economics strip (256/256 seats full, 217 earning miners, burn 0.0784 τ ≈ $18 at press time, alpha 0.0195 τ, 9.80 τ/day miner emission) and the golden-order banner with the gate numbers; below: the nine phase cards with copyable commands, cost badges and per-user progress counter, the Validator Lab gate card (41.9 weak vs 62.1 competitive), budget at a glance, and the honest-boundaries panel.

**Budget at a glance:** burn ~$12–18 one-off (irreversible, floats with seat demand — read the strip before executing) · hardware $0 — an existing CPU box is fine · expected net ~$100/mo base case. **Cadence:** the Part G.2 weekly routine applies here too, minus the pod checks — strip trend, judge re-mine after any subnet validator update, cohort spread. **Decision rule:** if the net expectation sits under your target for weeks straight, deregister and move the (small) burn elsewhere — that flexibility is the whole point of the cheap seat.

CHAPTER 7 · ~10 MINUTES · EXCHANGE-SIDE

# Funding from Binance — Click by Click

This is a plain deposit into your own wallet — no burn happens here. The only two ways to get this wrong are the **network** and the **address**, so both are double-checked below.

### B-1 Copy your coldkey address from btcli (never from memory)

Run `btcli wallet overview --wallet_name infranex` and copy the coldkey (¢) address — the one **not** labeled ℏ. Both start with 5; the ¢/ℏ marker is what tells them apart. If the marker is unclear, the address under your **coldkey** name is the funds wallet.

### B-2 In Binance: Spot → Withdraw → TAO

- **Step 1:** Binance app or web → **Assets → Spot → Withdraw** → search **TAO (Bittensor)**.
- **Step 2:** choose **Withdraw → Crypto**. Paste your coldkey address. Use **Binance mobile app** or web — both work identically.
- **Step 3: Network: choose "Bittensor (native)"**. This is the critical choice — TAO is its own chain. If a network dropdown shows anything else (a wrapped or bridged variant), stay on native Bittensor.
- **Step 4: No memo/tag is needed**. If Binance asks for one, you are on the wrong network — stop and re-check.
- **Step 5:** Amount: burn fee + ~2 TAO. Binance shows its withdrawal fee on the confirmation screen before you confirm.
- **Step 6:** Verify the **first 6 and last 6 characters** of the pasted address against the btcli output, then confirm (2FA). Arrival is typically minutes; Binance occasionally pauses TAO withdrawals during chain upgrades — the button greys out and resumes later.

### B-3 First time? Send a test amount first

Send 0.5 TAO, wait for it to land (check `btcli wallet overview` — the balance appears next to the coldkey), then send the rest. Costs one extra network fee; buys certainty on a chain you have never used.

YOUR LAPTOP · TERMINAL — AFTER FUNDING

```
$ btcli wallet overview --wallet_name infranex
Wallet: infranex   ✓- coldkey: 5CA1abcdef...  →  balance: 2.0493 TAO
```

**Three rules of exchange funding.** Native Bittensor network only. Coldkey address only (hotkeys are not for holding funds). Address check by first/last 6 characters every single time — there is no undo on a blockchain transfer.

CHAPTER 8 · WHEN SOMETHING LOOKS OFF

# Troubleshooting & Health Checks

The System & Errors view runs live checks on every dependency. It is also the first place to look when anything in the pipeline misbehaves — most incidents are one of the five below.

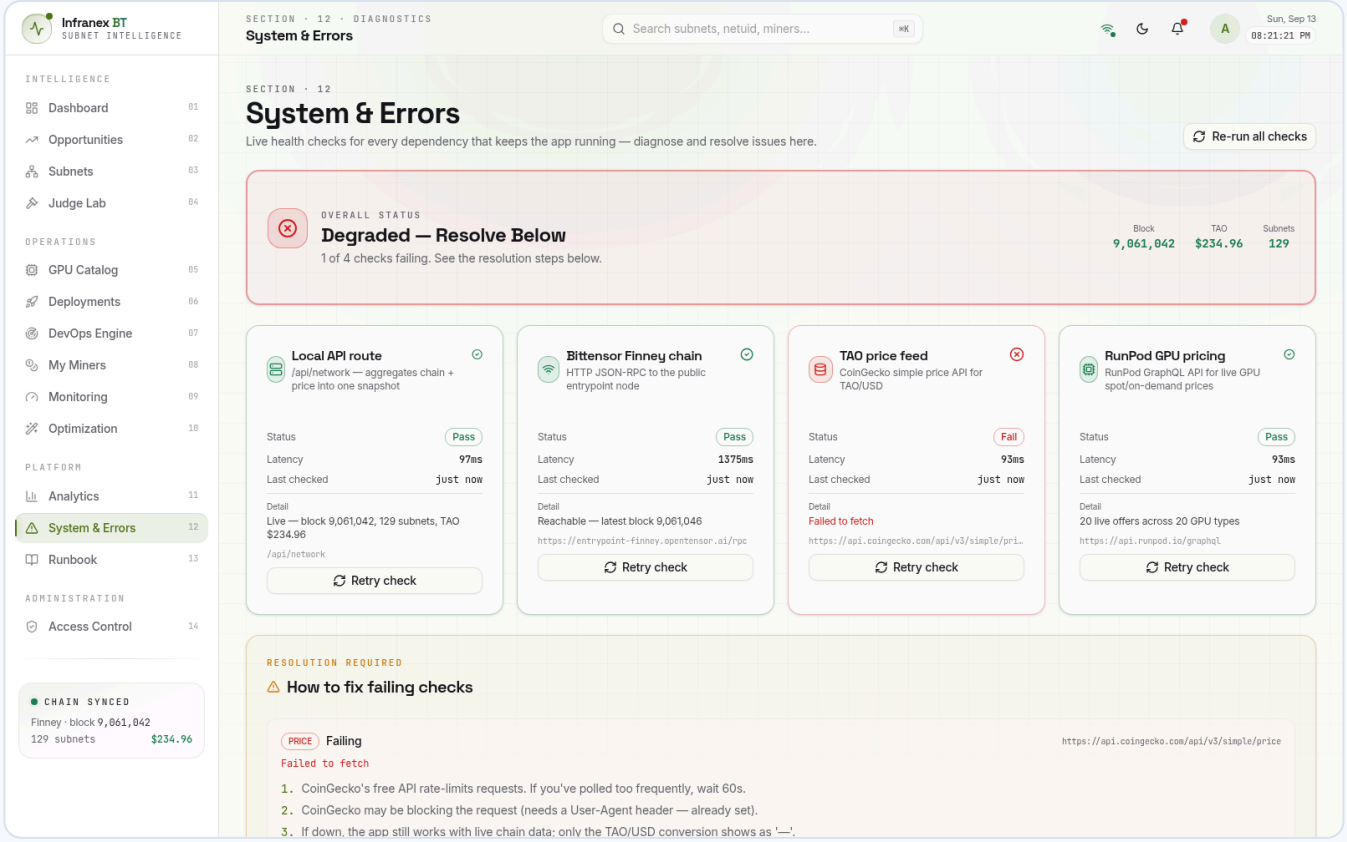



**Fig 8-1.** System & Errors health board. In this capture the TAO price feed is rate-limited by CoinGecko (transient — the platform keeps working on live chain data and self-heals on retry); chain, API and RunPod pricing are green.

| Symptom | Fix |
|---|---|
| **Verify says "Not registered yet"** | Registration takes one block to land (~12 s). Wait and click Verify again. Also confirm you registered with the same wallet/hotkey names you pasted. |
| **GPU step shows no offers** | No provider key saved, or the saved key lost validity. GPU Catalog → Provider API keys → re-paste and **Save & verify**. |
| **Install step stuck** | Open the deployment card's install log — each of the ~10 steps shows its output. Pod-boot delays up to ~2 min are normal on RunPod; a red step tells you exactly which script failed. |
| **Restart pending won't clear** | The restart runs over SSH to the pod. Check the pod is running (RunPod console) and the deployment's ssh port is reachable; then press **Restart miner** again. |
| **Emissions stay 0** | Normal for the first minutes — validators must probe your axon before scoring. Watch the incentive % on the UID Defense panel; if it stays flat for hours, check axon reachability from the Runbook panel. |

# You're mining.

Wallet created offline. Keys where they belong. GPU rented at the right second. Registered last. The platform watches the rest — spend, earnings, axon health, deregistration risk — every 90 seconds, with alerts when anything drifts.

- ✓ Coldkey mnemonic — on paper, in your safe, never online
- ✓ Hotkey file — on the pod only, via your own SCP
- ✓ RunPod key — encrypted at rest, audited on every change
- ✓ Registration burn — recorded on the deployment card
- ✓ P&L — real accruals from the first minute, not estimates
- ✓ CPU path — SN67 script mining available from any CPU, gated the same way (Part I)

Next moves when ready: scale to a second miner (same wallet, new hotkey — Part B again), or wire a real Slack/Discord webhook so alerts follow you. And when you terminate any pod — Terminate, never Stop.